

Pronóstico de series de tiempo (forecasting)

Sesión 3

Docente: Luis Garayar



Acuerdos para la sesión:

- **Puntualidad**

Se requiere puntualidad para un mejor desarrollo del curso.

- **Celulares en silencio**

Es necesario que los celulares permanezcan en silencio durante la clase para no interrumpir la sesión.

- **Preguntas y consultas**

Las preguntas pueden realizarse en cualquier momento. Las consultas muy particulares se revisarán al final de la clase.

- **Cumplimiento de tareas**

Realizar las actividades y/o tareas encomendadas dentro de los plazos establecidos.



Contenido

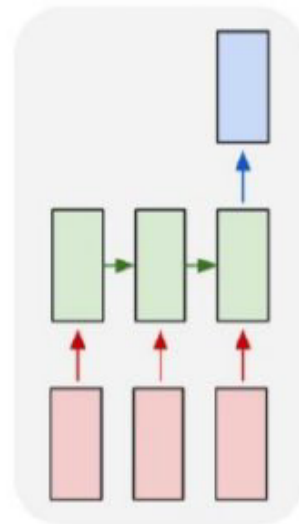


- Transfer Learning
- Limitaciones de las CNN y el porqué de las RNN
- Unidad Estructural y Funciones de RNN: La Neurona Recurrente
- Backpropagation a través del tiempo
- Exploding Gradients y Vanishing Gradients
- LSTM
- GRU

Deep Learning

RNN

- ▶ Hasta ahora, hemos visto dos tipos de redes neuronales: feed-forward y CNNs. En ambas, tenemos un input de tamaño fijo (por ejemplo, una imagen) y una única salida.
- ▶ En muchos problemas de machine learning nos gustaría tener más flexibilidad, con arquitecturas que tengan una **longitud variable** de inputs y de outputs.
- ▶ Por ejemplo: representación **bag-of-words** de un texto vs **secuencia de palabras**.



Deep Learning

RNN

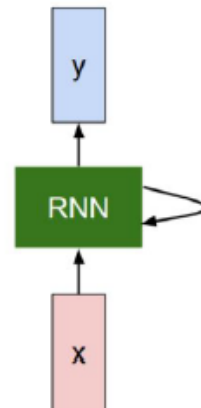
- ❑ ¿Cuál es la **limitación que tienen las CNN**?
 - Confían en la independencia entre los ejemplos de training y de test.
 - Confían en ejemplos de vectores de longitud fija.
 - Utilizan un conjunto de pasos de computación fijos (el número de capas del modelo).

- ❑ Necesitamos **modelizar los datos con estructuras temporales o secuenciales** y con **longitud variable de inputs y outputs**
 - Frames de videos
 - Snippets de audios
 - Palabras obtenidas de frases

Deep Learning

RNN

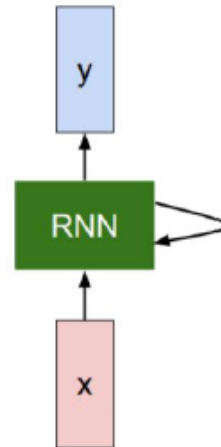
- ▶ Las **Recurrent Neural Networks (RNNs)**, o redes neuronales recurrentes, son un tipo de red neuronal capaz de trabajar con información secuencial.
- ▶ Mantienen un estado interno o **hidden state**, que podemos considerar como una memoria de lo visto hasta el momento.
- ▶ Aplican una fórmula recurrente sobre una secuencia de entrada de manera que, en cada paso de la secuencia, se depende del valor de input x de ese momento y del hidden state anterior h .
- ▶ Cada paso en la secuencia se conoce como *time step*.



Deep Learning

RNN

- ▶ La potencia de las RNNs radica en su capacidad de **modelar relaciones temporales** entre elementos de la secuencia a través del estado interno de la red.
- ▶ Esto explica por qué este tipo de arquitectura ha tenido gran éxito en problemas de **lenguaje natural**, donde el contexto y el orden son determinantes.
- ▶ Las redes recurrentes son muy versátiles y pueden aplicarse a varios tipos de problema según queramos secuencias de entrada y/o de salida.



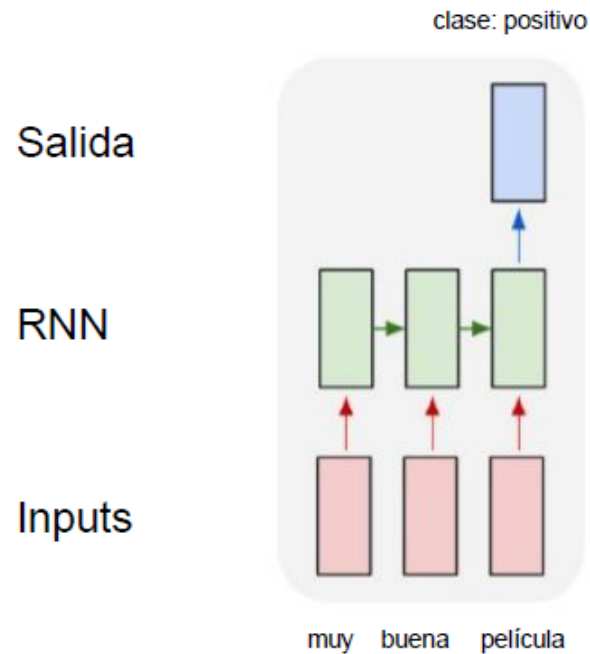
Deep Learning

RNN

Input: Secuencia

Output: Único output al final de la secuencia.

Ejemplo: **Análisis de sentimiento** de una oración o texto



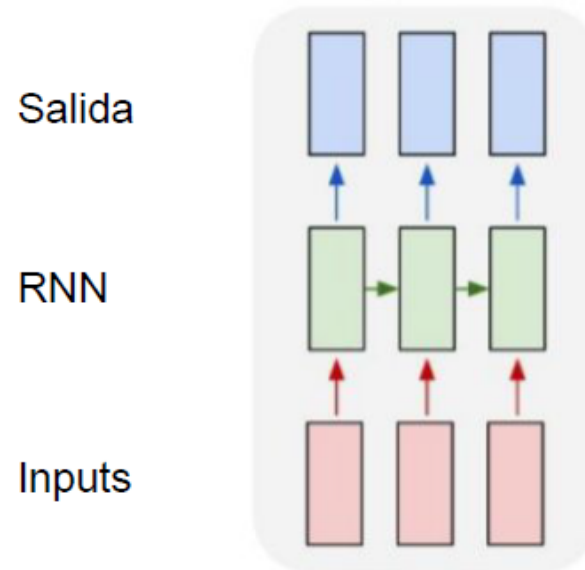
Deep Learning

RNN

Input: Secuencia

Output: Secuencia con un valor de salida por cada elemento de entrada.

Ejemplo: Clasificación de imágenes frame a frame en un vídeo.



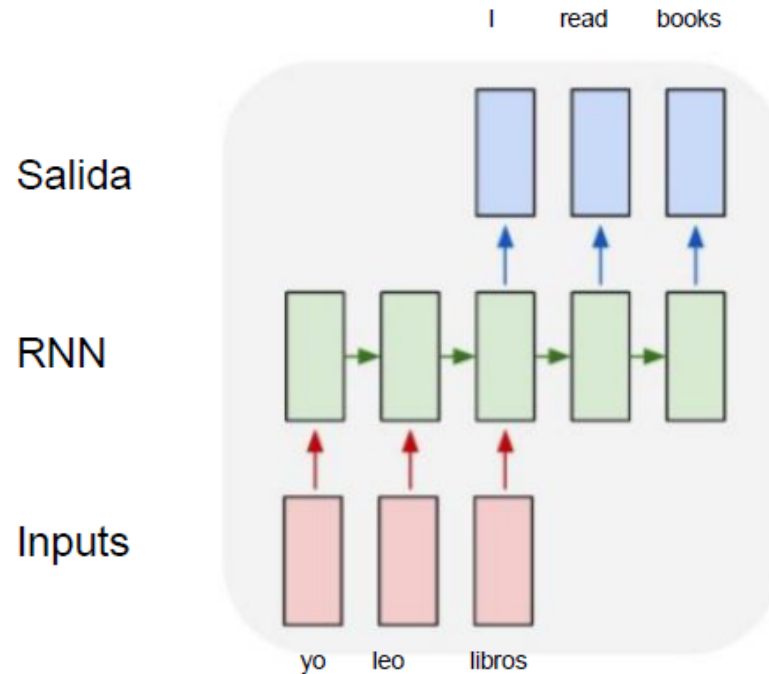
Deep Learning

RNN

Input: Secuencia

Output: Secuencia. Se espera a leer toda la secuencia de entrada antes de generar la secuencia de salida.

Ejemplo: Machine translation.



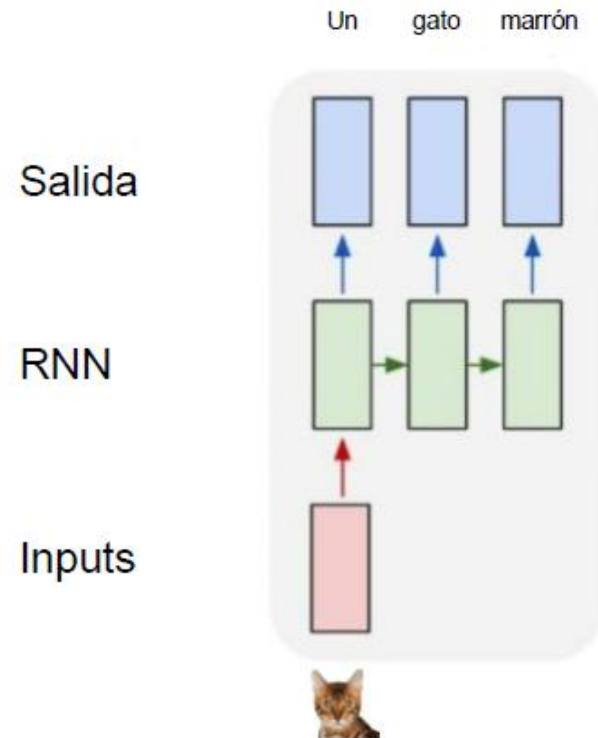
Deep Learning

RNN

Input: Elemento único.

Output: Secuencia.

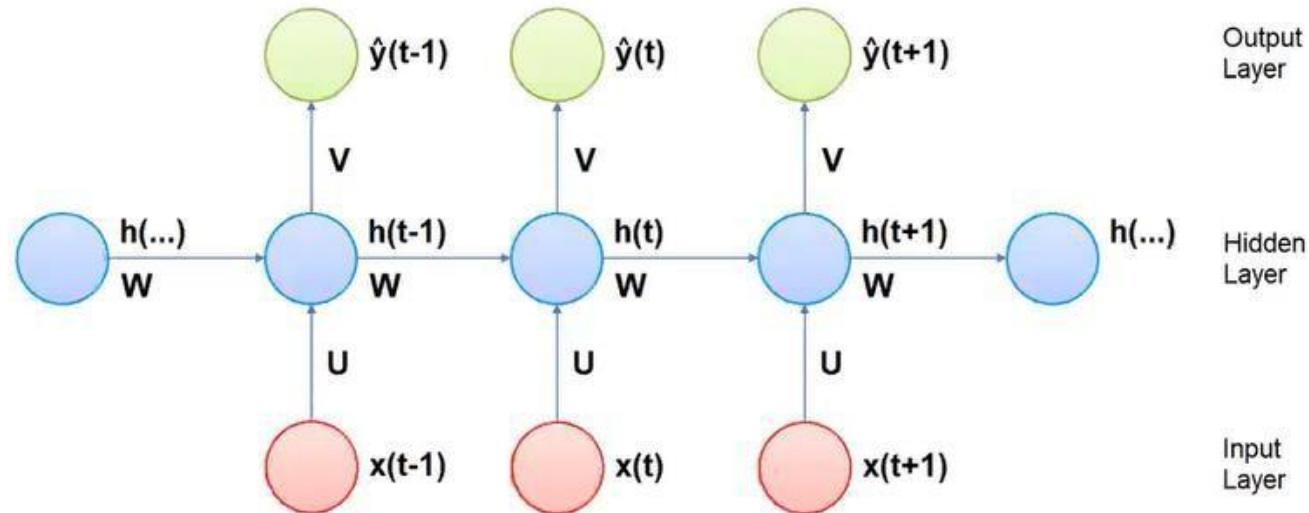
Ejemplo: Image captioning.



Deep Learning

RNN

Las Redes Neuronales Recurrentes (RNN) son **modelos conexionistas con la habilidad de pasar la información precisa entre pasos secuenciales.**



Input - The vector $x(t)$ is the input of the network at time step t .

Hidden state - The vector $h(t)$ is the a hidden state at time t , and is a sort of *memory* of the network; it is calculated based on the current input and the previous time step's hidden state:

$$h(t) = \tanh(W h(t-1) + U x(t))$$

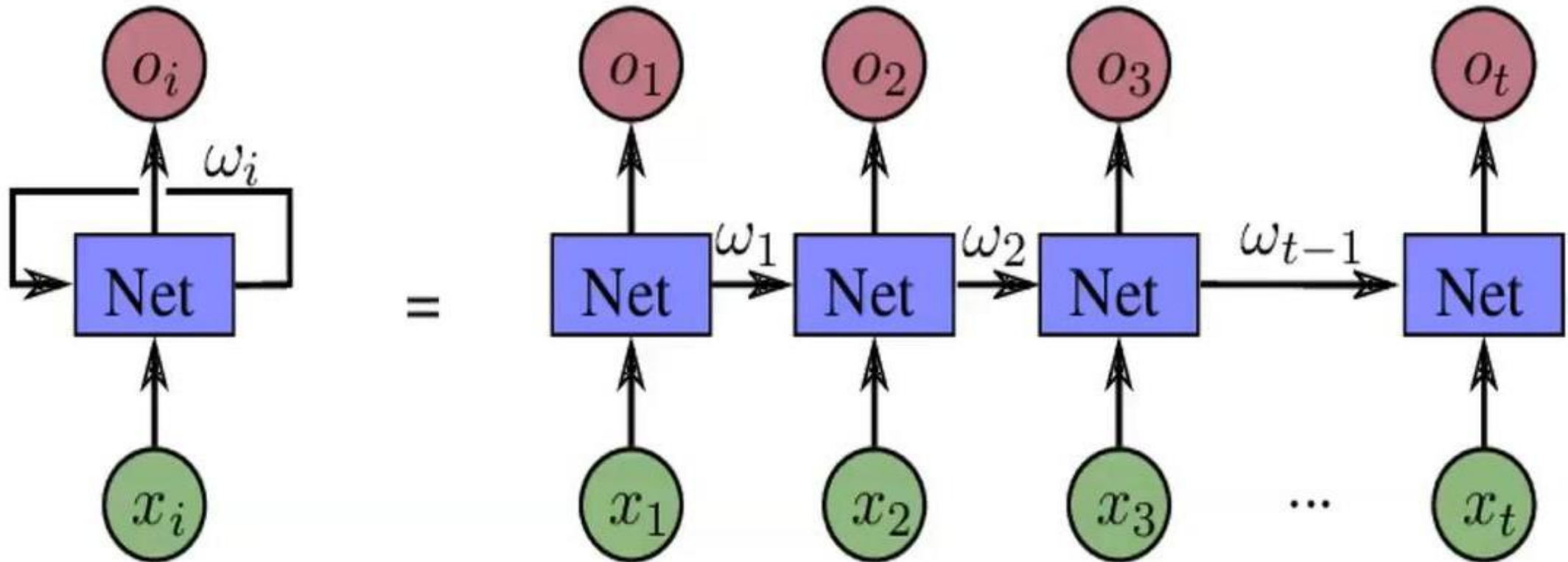
Output — The vector $\hat{y}(t)$ is the output of the network at time t :

$$\hat{y}(t) = \text{softmax}(V s(t))$$

Deep Learning

RNN

- Tienen **bucles** dentro, permitiendo que la **información persista** → **memoria**.



Deep Learning

RNN

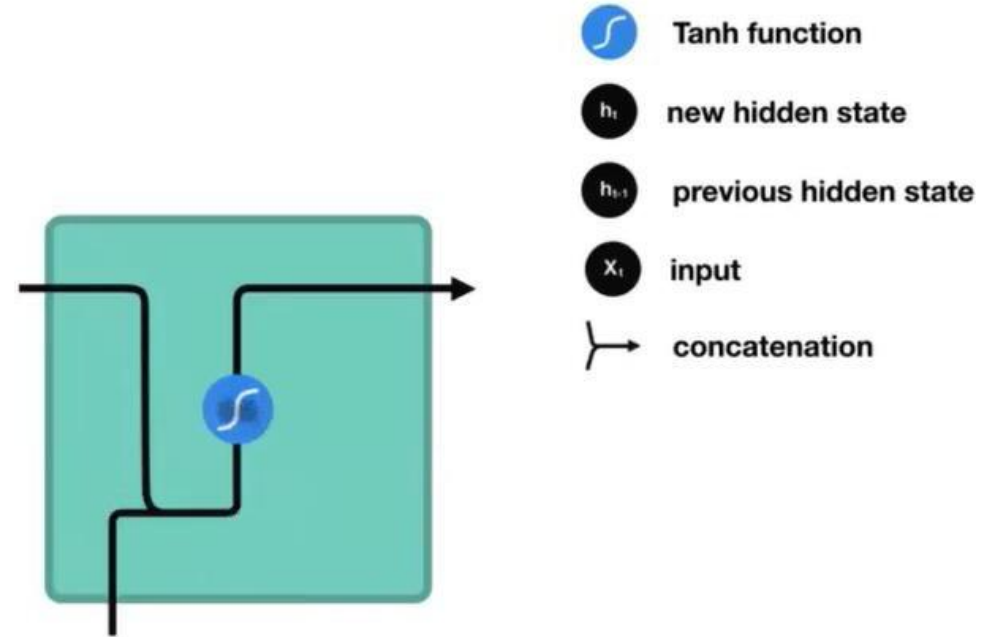
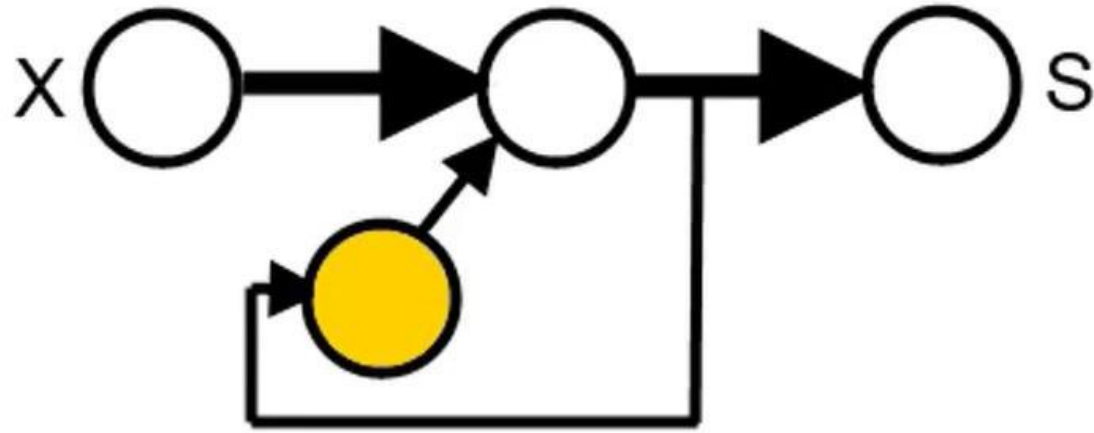
❑ Su arquitectura se basa en **cómo los humanos procesamos información:**

- Los **pensamientos humanos** tienen **persistencia**; no empezamos a pensar de cero cada vez
 - Según leemos, por ejemplo, esta frase, entendemos cada palabra a partir de lo que hemos entendido de las previas
- Uno de los elementos de valor fundamentales de las RNN es que son **capaces de conectar la información previa con la tarea actual**
 - Ejemplos
 - Usar frames de vídeo previos para conectar con el entendimiento del frame actual
 - Predecir la siguiente palabra a partir de las utilizadas anteriormente en una frase

Deep Learning

RNN

La Neurona Recurrente

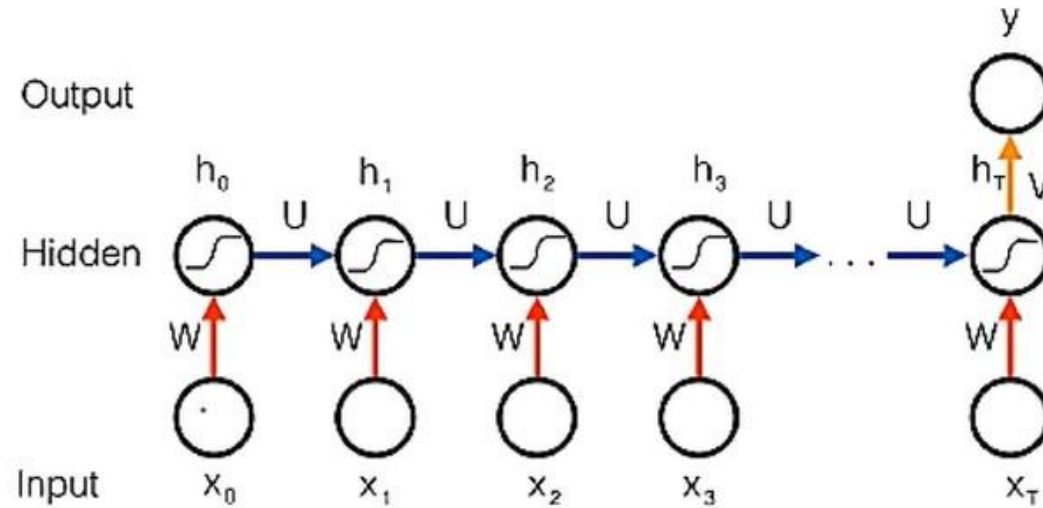


- ❑ La función de activación de las ANN solo actúa en una dirección, hacia delante, desde la capa de entrada hacia la capa de salida, es decir, que no recuerdan valores previos.
- ❑ Una red RNN es parecida, pero incluye conexiones que apuntan “hacia atrás”, una especie de retroalimentaciones entre las neuronas dentro de las capas.

Deep Learning

RNN

En cada instante de tiempo (***timestep***) esta neurona recurrente recibe la entrada x de la capa anterior, así como su propia salida del instante de tiempo anterior para generar su salida y .

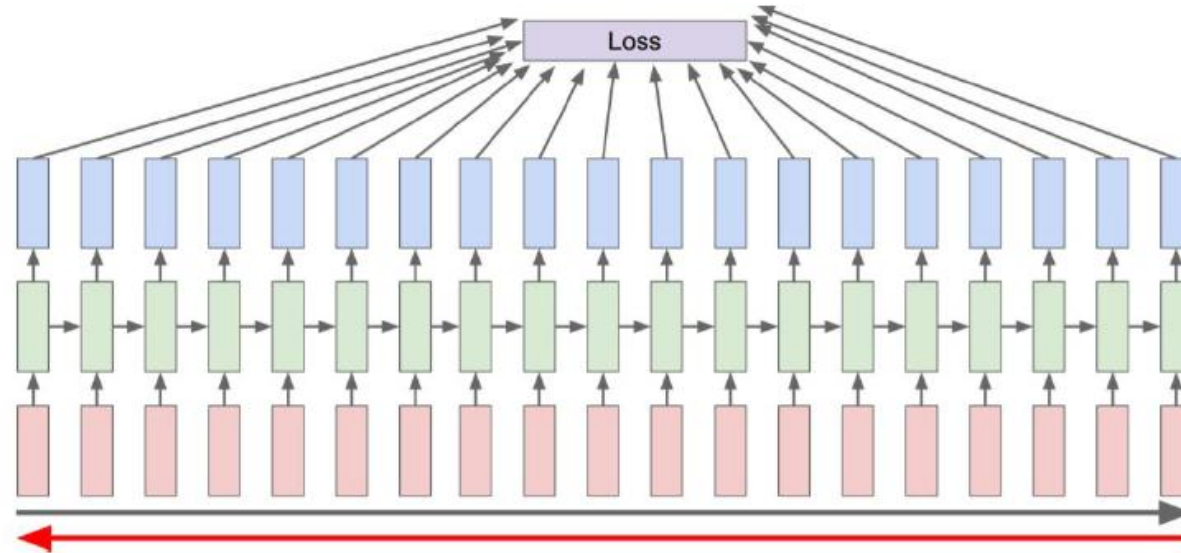


En **capa de neuronas recurrentes**, en cada instante de tiempo, cada neurona recibe **dos entradas**, la entrada correspondiente de la capa anterior y a su vez la salida del instante anterior de la misma capa.

Deep Learning

RNN

Backpropagation through time

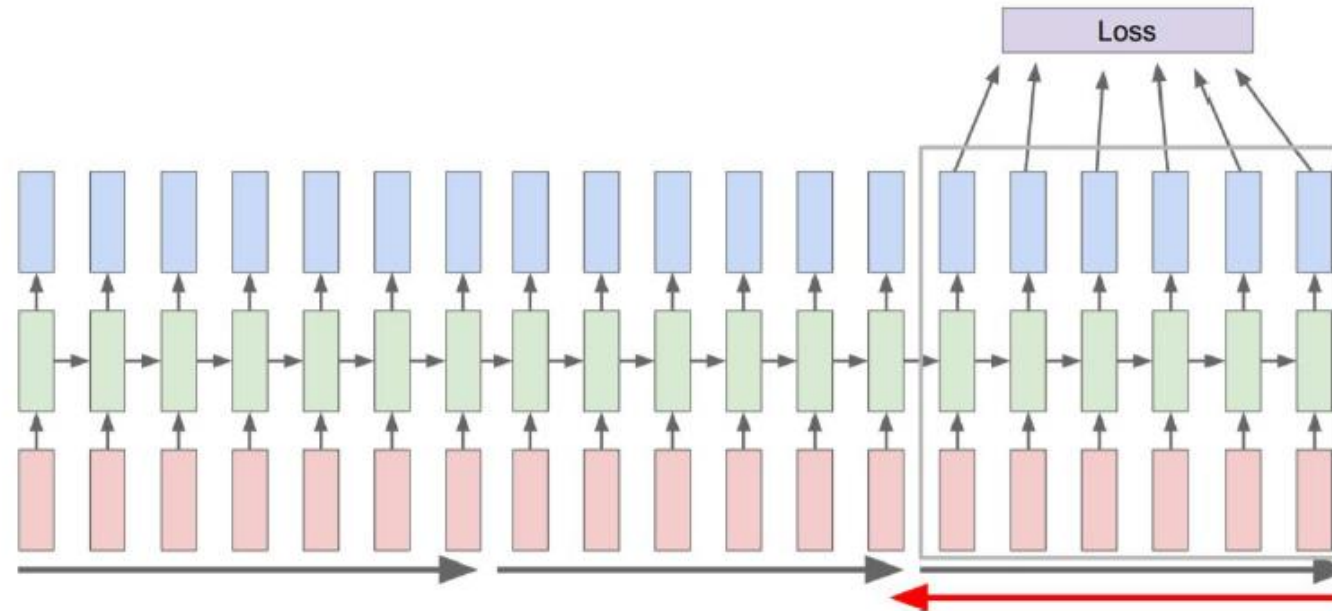


- ▶ Es necesario “desenrollar” el grafo de computación a lo largo del tiempo y aplicar backpropagation a partir de todos los outputs que hemos obtenido.
- ▶ **Backpropagation through time (BPTT):**
 - Se hace forward pass a lo largo de toda la secuencia para calcular la loss a partir de todos los outputs.
 - Se hace backward pass a lo largo de toda la secuencia.

Deep Learning

RNN

Backpropagation through time

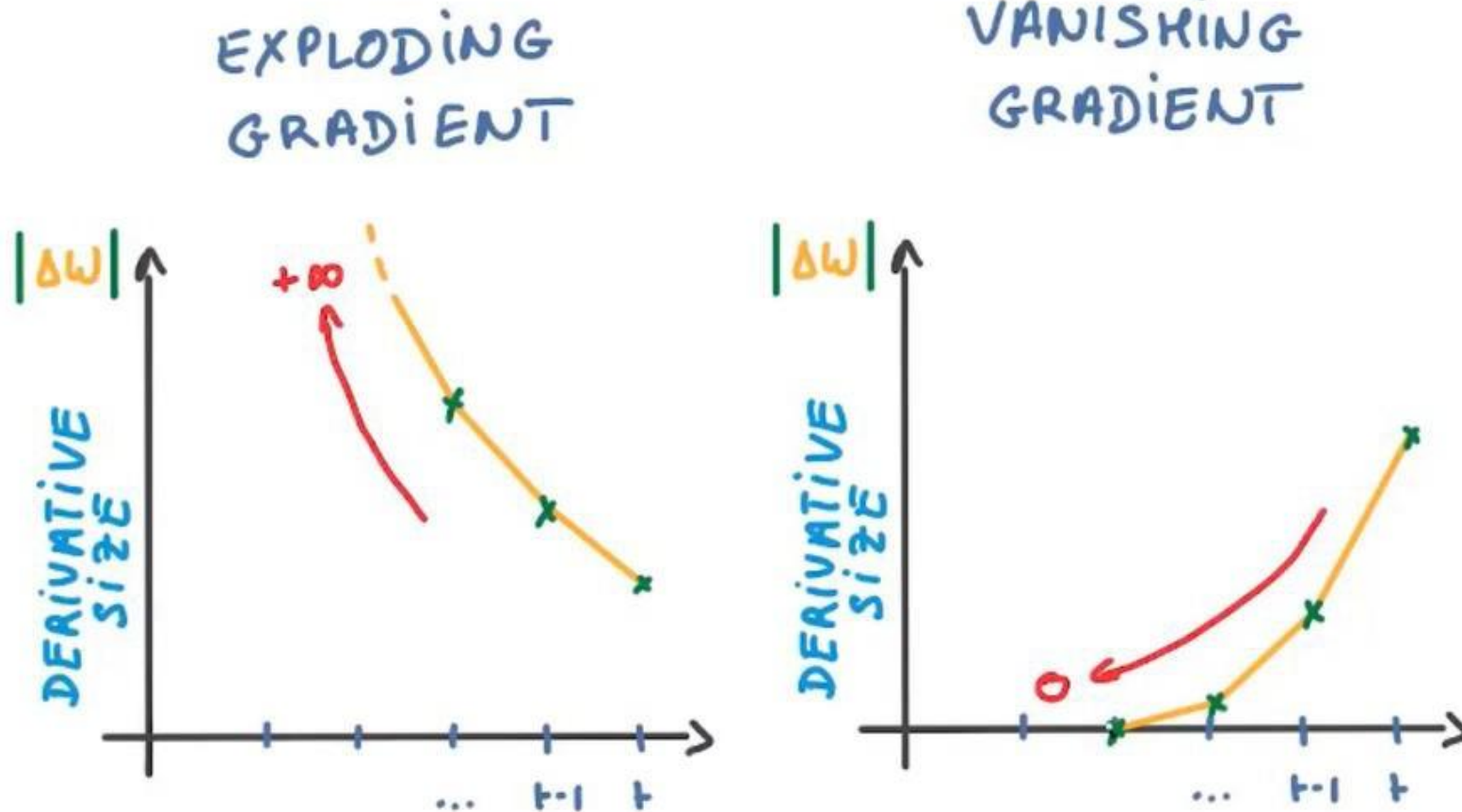


- ▶ BPTT es muy costoso para secuencias largas.
- ▶ En la práctica, se utiliza **Truncated Backpropagation through time**
- ▶ La secuencia se parte en trozos y se hace backpropagation en un pequeño número de steps, trozo a trozo.

Deep Learning

RNN

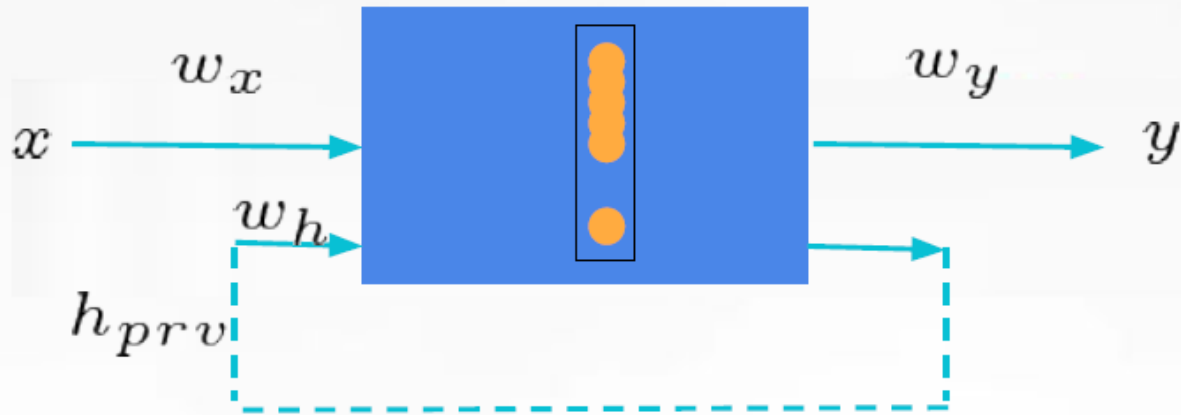
Exploding Gradients y Vanishing Gradients Through.



Deep Learning

RNN

Problemas en las RNN's



Debe recordar todos los Estados al mismo tiempo

Computacionalmente pesado

Fuertemente sensible a cambios en los parámetros

Vanishing Gradient

Exploding Gradient

Deep Learning

LSTM

Long Short-Term Memory Networks (LSTM)

- ❑ Las redes de memoria a largo y corto plazo (LSTM) se han desarrollado para superar el problema del gradiente de desaparición en el RNN estándar mejorando el flujo del gradiente dentro de la red. Esto se logra utilizando una unidad LSTM en lugar de la capa oculta.
- ❑ El estado de la celda transfiere la información relevante durante el procesamiento, de modo que también la información de los pasos de tiempo anteriores llega a cada paso de tiempo, reduciendo los efectos de la memoria a corto plazo. Durante el entrenamiento sobre todos los pasos de tiempo, las puertas aprenden qué información es importante mantener u olvidar, y la agregan al estado de la celda o la eliminan.
- ❑ De esta manera, LSTM permite la recuperación de los datos transferidos en la memoria, resolviendo el problema del gradiente que desaparece. Los LSTM son útiles para clasificar, procesar y predecir series de tiempo con retrasos de duración desconocida

Deep Learning

LSTM

- ▶ **LSTM: Long Short-Term Memory**. Datan de 1997.
- ▶ En la práctica, las redes recurrentes simples presentan problemas para aprender relaciones entre elementos en *time steps* muy separados, debido a factores como los *vanishing* y *exploding gradients*.
- ▶ Estos problemas hacen que parte del potencial de las redes recurrentes se pierda.
- ▶ Las **LSTM** se diseñaron para solucionar este problema de “memoria” de las **vanilla RNN**.
- ▶ Mantienen un estado interno adicional, el **cell state** (c_t), aparte del tradicional **hidden state** (h_t).

Deep Learning

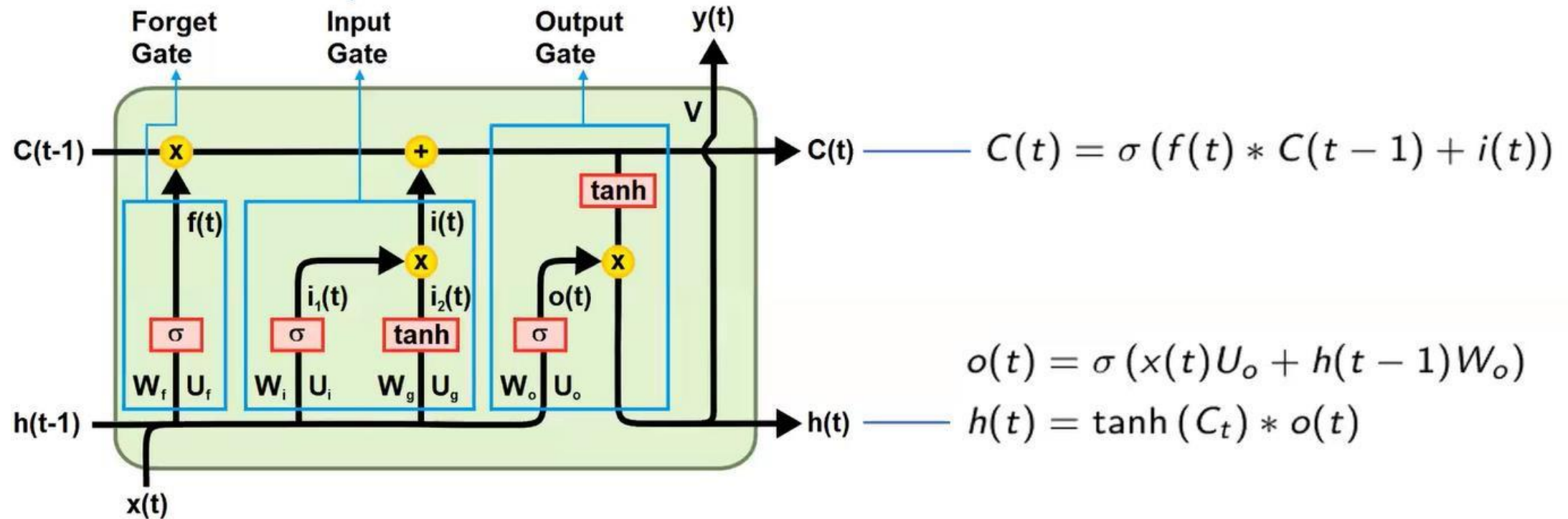
LSTM

$$f(t) = \sigma(x(t)U_f + h(t-1)W_f)$$

$$i_1(t) = \sigma(x(t)U_i + h(t-1)W_i)$$

$$i_2(t) = \tanh(x(t)U_g + h(t-1)W_g)$$

$$i(t) = i_1(t) * i_2(t)$$



Deep Learning

LSTM

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

i, f, o son vectores con valores entre 0 y 1. Actúan de “puertas”

Cuatro vectores distintos que nos dan los valores de c_t y de h_t : i, f, o, g .

- f se conoce como **forget gate** y, al ser multiplicada por c_{t-1} , nos dice cuánto tenemos que olvidar del estado anterior.
- i se conoce como **input gate** y representa cuánto hemos de escribir en c_t a partir de lo visto en g (sin nombre en especial)
- c_t se obtiene como una mezcla de “cuánto recordamos del pasado” ($f \cdot c_{t-1}$) y “cuánta información nueva queremos añadir” ($i \cdot g$).
- o se conoce como **output gate** y nos dice cuánto de nuestro estado interno c_t hemos de revelar en el nuevo *hidden state*.

Deep Learning

Aplicaciones de las RNN

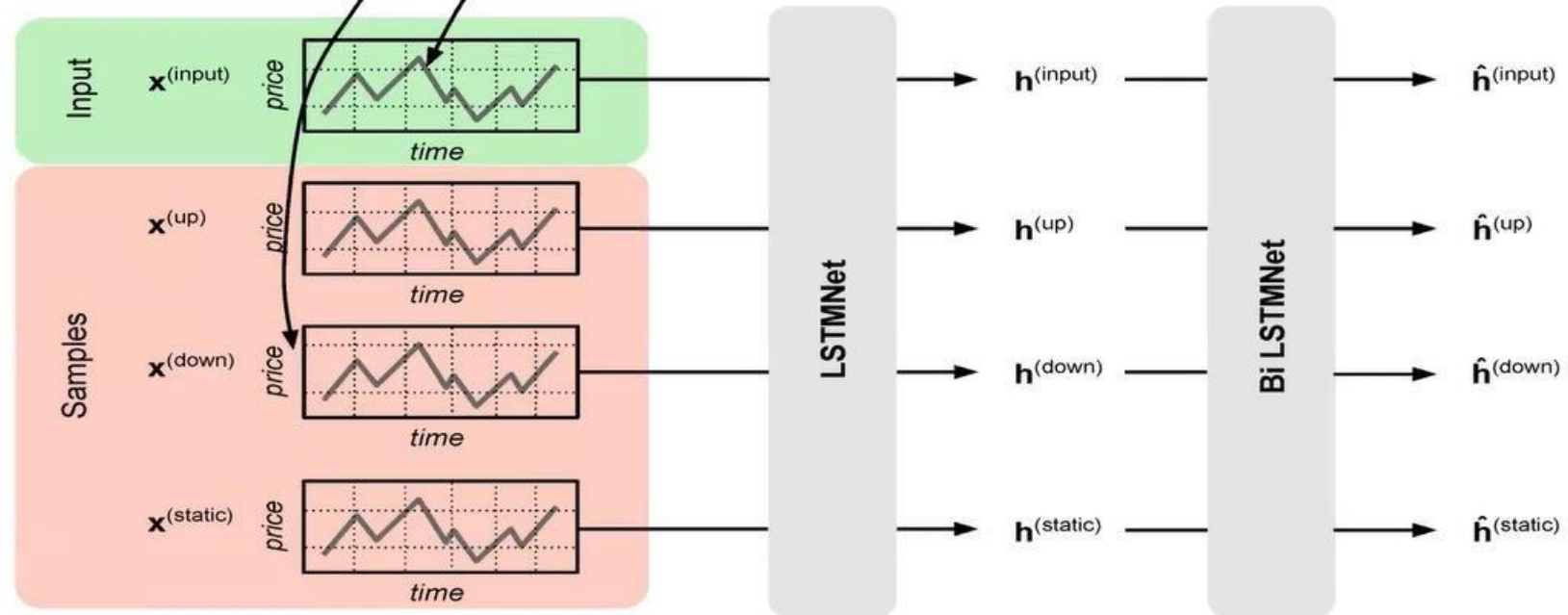
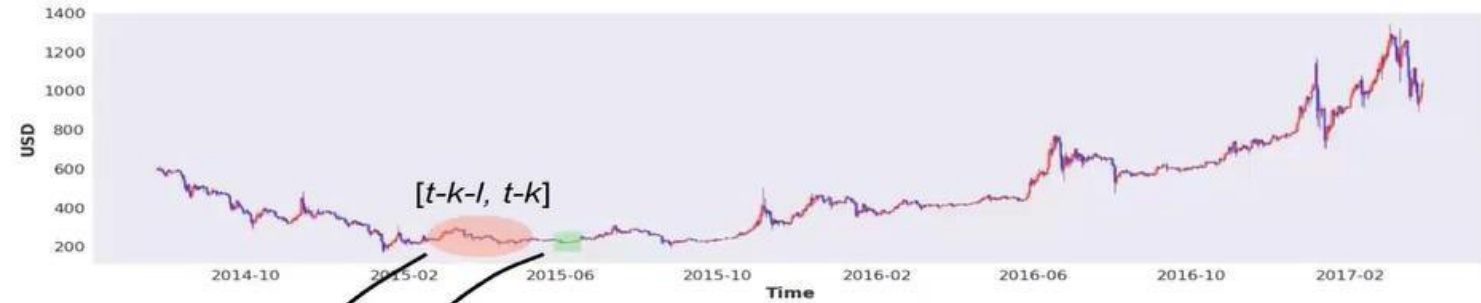
- ❑ Han tenido un éxito sorprendentemente alto modelando el lenguaje.

Task	Input Sequence	Output Sequence
Machine translation (Sutskever et al. 2014)	English	French
Question answering (Bordes et al. 2014)	Question	Answer
Speech recognition (Graves et al. 2013)	Voice	Text
Handwriting prediction (Graves 2013)	Handwriting	Text
Opinion mining (Irsoy et al. 2014)	Text	Opinion expression

- ❑ Las redes neuronales **Recursivas** pueden entenderse como una **generalización** de las RNN.
 - **RNN** → estructura lineal
 - **Recursive NN** → estructura jerárquica

Deep Learning

Aplicaciones de las RNN

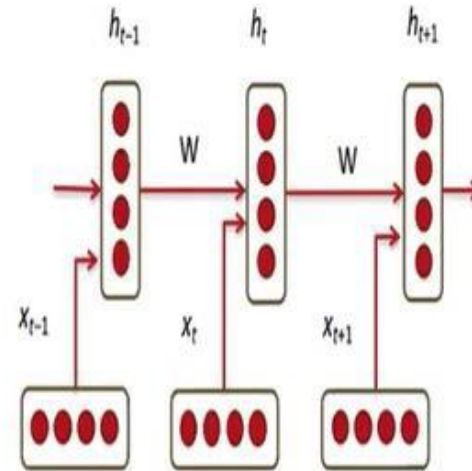


Deep Learning

Aplicaciones de las RNN

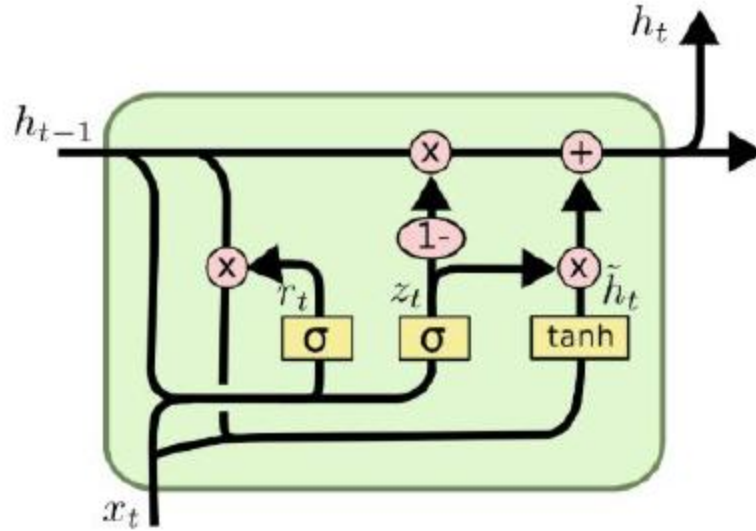
Machine Translation, Resumen Textos, Completar frases o discurso.

Gran importancia en áreas como el procesamiento del lenguaje natural, traducción automática, *speech recognition*...



$$\log p(o|c) = \log \frac{\exp(u_o^T v_c)}{\sum_{w=1}^W \exp(u_w^T v_c)}$$
$$X_{shirt} - X_{clothing} \approx X_{chair} - X_{furniture}$$
$$X_{king} - X_{man} \approx X_{queen} - X_{woman}$$

DEEP LEARNING



$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$$

$$z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$$

$$\tilde{h}_t = \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h)$$

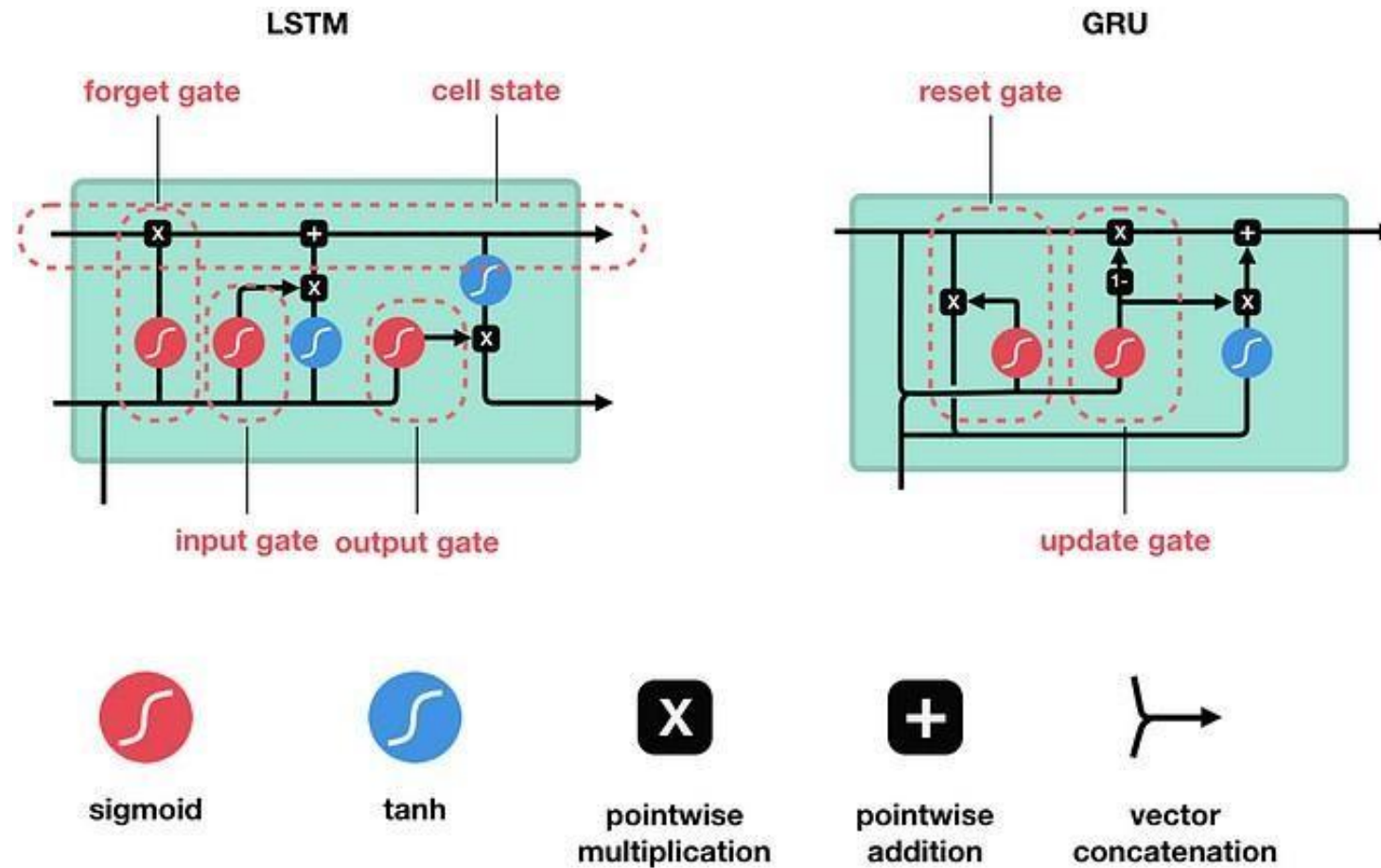
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

- ▶ **GRU - Gated Recurrent Unit.** Introducida en 2014.
- ▶ Tiene un sistema similar de *gates* al visto en la LSTM.
- ▶ Combina el cell state y el hidden state en un solo elemento.

DEEP LEARNING

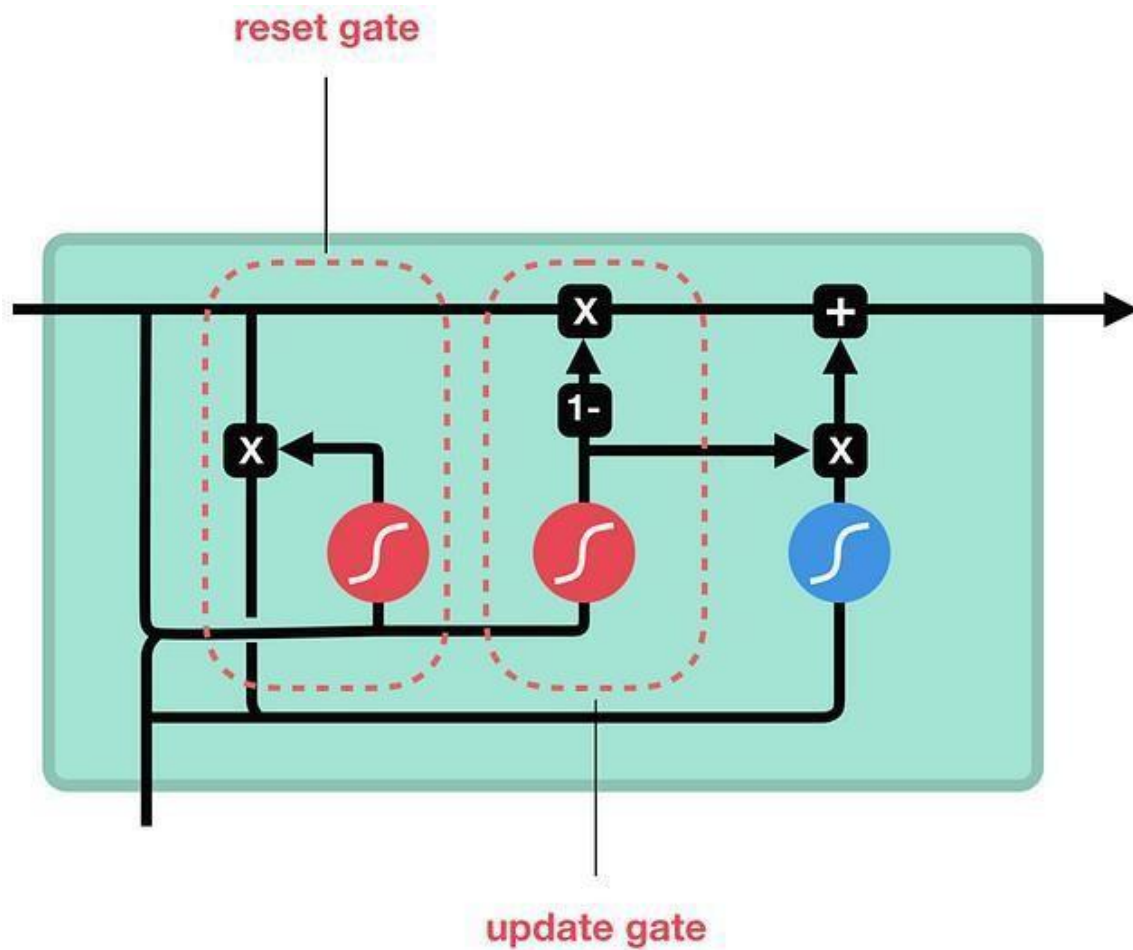
GRU

GRU es la nueva generación de redes neuronales recurrentes y es bastante similar a un LSTM. GRU se deshizo del estado de la celda y utilizó el estado oculto para transferir información. También solo tiene dos puertas, una puerta de reinicio y una puerta de actualización.



DEEP LEARNING

GRU



Update Gate

La puerta de actualización actúa de manera similar a la puerta de entrada y olvido de un LSTM. Decide qué información desechar y qué información nueva agregar.

Reset Gate

La puerta de reinicio es otra puerta que se utiliza para decidir cuánta información pasada olvidar.

GRU tiene menos operaciones tensoriales; por lo tanto, son un poco más rápidos de entrenar que los LSTM. No hay un ganador claro cuál es mejor. Los investigadores e ingenieros suelen probar ambos para determinar cuál funciona mejor para su caso de uso.



¡GRACIAS!

